Question 7 [300 marks]

THE ARACANUM EDITOR

The year is 2009. Blackwood and Hargrove is one of the last independent publishing houses in London still operating out of their original Clerkenwell offices. It is a converted Victorian printing warehouse where the radiators clang at odd hours and the basement still smells of ink. They have survived the digital revolution by sheer stubbornness and a devoted readership, but their infrastructure tells a different story. Their editorial servers are a cluster of early 2000s Windows machines tucked into a cage behind the mail room. They have no graphical desktops. Just blinking cursors on black screens, connected over a painfully slow internal network.

For decades, their editors worked with a bespoke DOS tool built by a contractor who vanished sometime around 2003. That tool got corrupted beyond repair after a power surge and finally died last Tuesday, taking down the only software that understood Blackwood and Hargrove's peculiar manuscript format. Commercial word processors won't run on the servers. Lightweight text editors can't handle their peculiar demands.

They now want you to build a windows specific app for them using pure c++ and WinAPI calls. A starter code in editor.cpp has been provided to you with the WinAPI console scaffolding. Your job is to bring it to life.

A Blackwood and Hargrove manuscript is not a continuous stream of text but rather structured data in an output of a specific format.

Every document is a collection of pages. Each page is divided into **n** number of columns, laid out side by side. Each column contains a fixed number of lines, and each line holds a fixed number of characters. These numbers should first be prompted and inputted when the program is first opened to provide a custom user experience; so that each user could choose according to their specific needs. Sensible defaults might be 20 lines per column and 40 characters per line, but the design should make it easy to adjust; even in the midst of running the program.

The most important rule of this format is word integrity. When text flows into a column, words must never be broken across lines. Suppose a line can hold 10 characters, and you're inserting the text `"Hello Everyone"`. A naive approach would split it like this:

```
Hello Ever
yone
```

That's unacceptable. The word `"Everyone"` must move to the next line in its entirety:

```
***Hello***
***Everyone***
```

When a column fills up, text overflows into the right column of the same page. When both columns on a page are full, a new page is created automatically. All of this should be done automatically; the writer should never have to think about pagination or column breaks. They type, and the text appears in the right place.

It is an interactive, real-time console editor. The screen should always display the current state of the document; the two columns of the active page, a status bar, and any contextual information (search highlights, word count, current alignment mode and so on).

Input is handled character by character using WinAPI console input functions. When the user presses a key, the editor responds immediately. There is no "press Enter to submit" for navigation commands. The user could type word by word and it should display in real time; or the user could copy paste large chunks of data and it should display it out just like a text editor would immediately register it and apply the Pagination and so on.

Feature Specifications

The following features should be fully implemented onto the application; apart from the layout requirements as prescribed earlier.

Search and Highlight

When the user initiates a search, every occurrence of the search term on the **current page** must be visually distinguished from normal text. You can use WinAPI console attributes to change the foreground or background colour of matching regions

The editor must also maintain a **search history**; the last five unique search terms, each annotated with the number of matches found. This history should be viewable through a dedicated command. Searching is case-insensitive.

Text Alignment

Different parts of a manuscript require different alignment:

- Left-aligned: the default. Text is flush against the left margin, ragged on the right.

- Right-aligned: text is flush against the right margin. Used for dates, signatures, and attributions.
- Centre-aligned: text is centred within the column width.
- Justified: text is stretched so that both the left and right margins are flush. This is achieved by distributing extra spaces between words. The last line of a justified paragraph should remain left-aligned .

When the user sets an alignment mode, all subsequent paragraphs are formatted with that alignment until the mode is changed. The alignment is dynamic and can be changed during runtime and so subsequent changes are applied throughout.

Automatic Table of Contents

How does the editor generate a Table of Contents when a document contains no explicit headings or structural markers? The editor scans the first meaningful paragraph of each page and runs it through a score-based heuristic. Lets not get confused by the word heuristic. Let's try to demystify it (you'll thank me in your later courses). Think of it as a point scoring system and we award them based on some 'if' statements. Once all points are accumulated, we then check it against a threshold. If it were greater than that threshold, then we make a decision else we do not. Think of it as a systematic or informed guessing; the kinda stuff you do during exams selecting MCQs you aren't sure about.

Points are awarded across several criteria. A word count of eight or fewer earns two points, because chapter titles are rarely that long. A character length under sixty earns one point. The absence of terminal punctuation such as a period, question mark, or exclamation mark earns two points, since titles typically do not close like sentences. Capitalization pattern earns two points when most words begin with an uppercase letter and the text does not read as a fully lowercase block, catching patterns like ***The Locked Study*** **or** ***A Matter of Inheritance***. The absence of commas earns one point, since chapter titles rarely contain them. Each paragraph is evaluated against all five criteria and assigned a cumulative score. If that score reaches five or above, the paragraph is accepted as the chapter heading and entered into the Table of Contents with its page and column reference.

To keep the detection reliable and reduce false positives, the heuristic is applied only to the first paragraph on a page. A paragraph that is not preceded by a blank line is skipped entirely, since mid-flow text is unlikely to be a heading regardless of how it scores.

If no paragraph clears the threshold, the editor falls back to extracting the first four words of the opening paragraph. If the last of those words is a common stop word such as *was*, *is*, *and*, or *the*, it is dropped. The remaining words are capitalized and used as the stand-in title. The editor also accounts for edge cases: empty pages, pages that open mid-sentence from a continuing chapter, pages that begin with dialogue, and pages whose first paragraph runs unusually long. In all these cases the fallback ensures a Table of Contents entry is still produced.

A sample of how it would look like is described below

```
TABLE OF CONTENTS
──────────────────────────────────────────────────────────────
1. The Disappearance ..................... Page 1
2. Inspector Ashworth ................... Page 2
3. The Locked Study ..................... Page 3
4. A Matter of Inheritance ............ Page 5
```

Multi-Document Tabs

The editor must support up to 10 documents open simultaneously; but make sure the user can’t instantiate an 11th instance. This should be enforced at a class level structure, even if main lets say calls an 11th instance, it shouldn’t be made. Remember each document tab is an independent instance.

The user cycles through tabs with Ctrl+Tab (forward) and Ctrl+Shift+Tab (backward). The status bar always shows the current tab number and the total number of open tabs (e.g., `"[Doc 3/7]"`). There should be commands to open a new document, close the current tab, and save the current tab to a file.

Bonus
Memory management matters here. If all 10 tabs are in use and each contains a large manuscript, memory usage could become significant. Use lazy loading where possible; if a tab hasn't been viewed recently, you might store only its file path and reload from disk when the user switches to it.

Auto-Save

Use appropriate file management techniques to enable this.

The editor should automatically save the current document at a configurable interval (default: every 60 seconds). It should maintain last 10 copies of autosave and discard the previous ones.

Bonus
To make things easier for you, instead of writing the entire document to disk each time, compute and save only the **delta** i.e the difference between the current state and the last saved state.

Maintain the last 10 deltas on disk. If the editor crashes, the document can be reconstructed by loading the last full save and replaying the deltas in order. Each delta file should contain enough information to move forward (apply the change) or backward (reverse the change).

When the user explicitly saves (not auto-save), write a full snapshot and reset the delta chain. This keeps the recovery system simple and bounds the number of deltas that need to be replayed.

Spell Checker

Load a dictionary file as provided to you with the question. As the user types, check each completed word against the dictionary. Misspelled words should be visually marked; using a distinct console colour attribute (for example, red foreground or a red underline character beneath the word).

Just a props up, be efficient with your traversal as reading a file of this scale can instantly reap a havoc.

Live Word Count Statistics

Display a persistent statistics panel showing real-time metrics.

- Characters (with and without spaces)
- Words
- Sentences
- Pages (based on the document's actual page count)
- Estimated reading time (assume an average reading speed of 200 words per minute)

These statistics update instantly as the user types or performs any modification to the text.

Mouse Support

Using the WinAPI console input functions, implement full mouse support:

- Click to position the cursor within the text.
- Click and drag to select a region of text.
- Right-click for a context menu offering common operations (cut, copy, paste, search, alignment).

Mouse input arrives through the standard message loop inside WndProc. Handle *WM_LBUTTONDOWN* for click-to-place, *WM_MOUSEMOVE* combined with the MK_LBUTTON flag for click-and-drag selection, and *WM_RBUTTONDOWN* to trigger the context menu via *TrackPopupMenu*. In all three cases, pixel coordinates are extracted from *lParam* using *LOWORD(lParam)* for x and *HIWORD(lParam)* for y, which are then translated into document positions page, column, line, and character offset; based on your font metrics and layout geometry.

You may add further mouse support features as a bonus improving the UI, and making it easier for you to demonstrate your assignment effectively.

Auto-Numbering and Footer

Every page should display a footer at the bottom containing the page number, centred within the page width. The format should be:

```
— 1 —                — 2 —                — 3 —
```

Page numbers are generated automatically and update when pages are added or removed. The footer is not part of the document's editable content; it is rendered by the display engine and does not occupy any of the column's usable lines.

BONUS : Split Window Views

The editor should support splitting the screen into two panes, either horizontally (top/bottom) or vertically (left/right). Each pane can display a different part of the same document. For example, the writer might want to reference Chapter 1 while editing Chapter 5.

Each pane has its own independent cursor and scroll position, but they share the same underlying document. Edits made in one pane are immediately visible in the other.

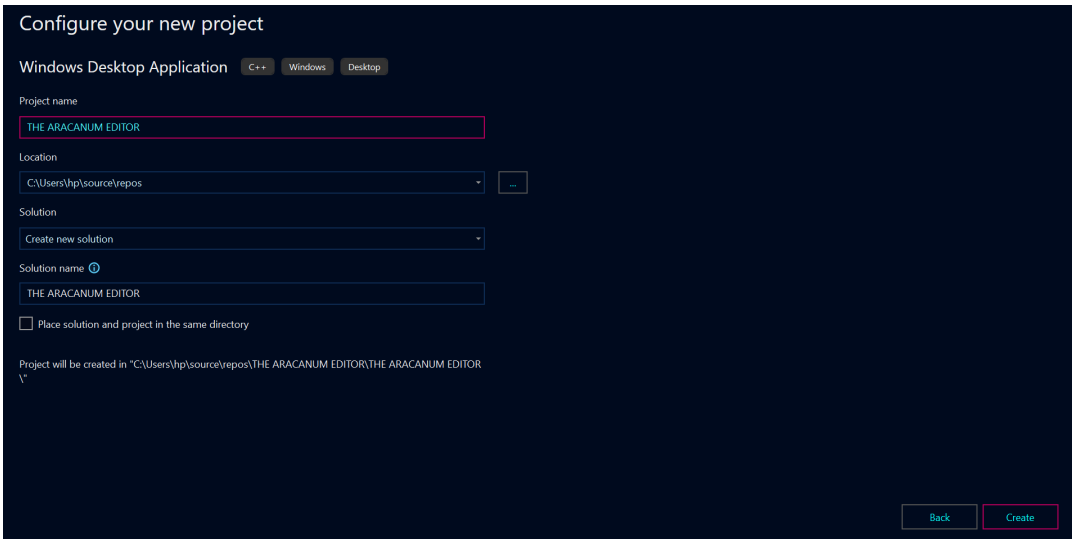
The user should be able to toggle the split on and off, switch focus between panes, and adjust the split ratio.

IMPLEMENTATION and Compilation GUIDELINES:

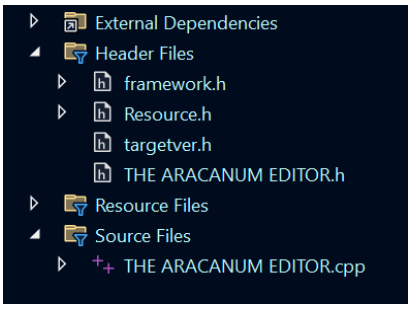
A editor.cpp has been provided which includes the started code on implementing a WinAPI GUI based app. Make sure you follow the following steps on Visual Studio to Create a Project.

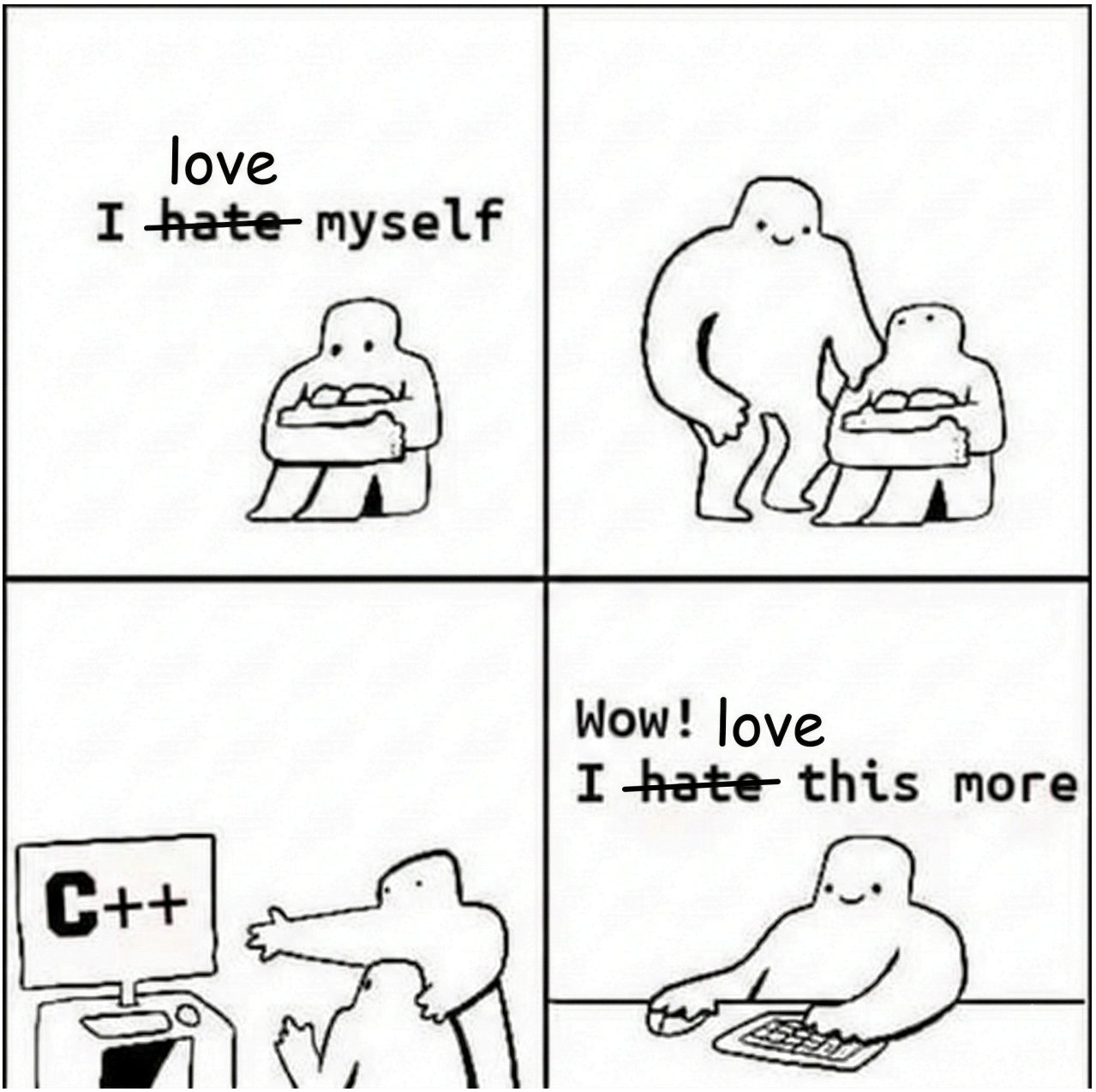
Step 1: Go to ‘Create a new Project’
Step 2: Search up ‘Windows Desktop Application’ and select it(the top option) as shown below

Step 3: Name it accordingly and create it as shown below



Step 4: The file structure shall be similar to this. You may understand the function calls from what the default code Visual Studio provides. You can start working. For the starter code provided alongside, kindly paste it fully onto the *.cpp* file and Run the Project





BEST OF LUCK

Eid Mubarak in Advance!!!